

3.3 Flujo de trabajo de un proyecto de Machine Learning

M. Sc. Huáscar Fedor Gonzales Guzmán · Carrera de Ingeniería Informática · UATF · Gestión 2026-02

En los temas 3.1 y 3.2 aprendimos a construir un modelo: definimos una hipótesis, una función de costo y un método de optimización. Pero en la práctica el modelo es solo **una pieza** de un proceso más amplio. Antes de entrenarlo hay que saber qué problema resolvemos y con qué datos; después hay que medir si sirve, mejorarlo y explicar sus resultados a quien tomará decisiones. Este tema presenta ese recorrido completo: el **flujo de trabajo** de un proyecto de machine learning. La idea central es sencilla pero exigente: *un proyecto puede fracasar aunque el código funcione*.

3.3.1 Visión general del flujo

Todo proyecto supervisado recorre, con más o menos detalle, ocho etapas. Cada una responde a una pregunta concreta y entrega un producto que alimenta a la siguiente:

#	Etapas	Pregunta que responde	Producto
1	Definir el problema	¿Qué decisión queremos apoyar y qué debe producir el modelo?	Tarea de ML bien formulada
2	Conseguir y explorar datos	¿Los datos disponibles permiten responder la pregunta?	Dataset comprendido y diagnosticado
3	Elegir variables	¿Qué información verá el modelo al predecir?	Matriz X y vector y
4	Separar train / test	¿Cómo simularemos datos que el modelo nunca vio?	Conjuntos de entrenamiento y prueba
5	Entrenar	¿Qué patrones aprende el modelo de los ejemplos?	Modelo ajustado (una hipótesis)
6	Evaluar	¿Funciona bien en datos nuevos y mejor que algo simple?	Métricas y análisis de errores
7	Mejorar	¿Qué cambio fundamentado reduciría los errores que importan?	Nueva versión documentada
8	Comunicar	¿Qué significa el resultado y cuáles son sus límites?	Informe o presentación

NOTA • **El flujo no es una línea recta**

Las etapas se presentan en orden, pero el trabajo real es **iterativo**. Explorar los datos puede obligarnos a reformular el problema; evaluar puede enviarnos de vuelta a elegir variables. Volver atrás no es un fracaso: es la forma normal en que se aprende sobre el problema. Lo importante es que cada vuelta esté guiada por lo que descubrimos en la anterior, y no por cambios al azar.

3.3.2 Definir el problema

El primer paso ocurre antes de abrir el notebook. Un error muy frecuente es empezar por el dataset o por el algoritmo que queremos probar, y recién al final preguntarnos qué buscábamos. Un modelo no tiene sentido por sí mismo: lo tiene en relación con **una pregunta, una necesidad o una decisión**.

a) De la necesidad a la tarea

Las necesidades reales llegan en lenguaje amplio: «mejorar las ventas», «reducir la deserción», «detectar riesgos». Ninguna de ellas es todavía un problema de ML. Definir el problema es **traducirlas** a una tarea concreta: clasificar, estimar un número, agrupar o detectar anomalías (tipos que vimos en la Unidad 1). Una misma necesidad admite varias traducciones, y cada una implica datos, modelos y métricas distintos.

b) Unidad de análisis y momento de la predicción

Hay que decidir **qué representa cada fila** —una persona, una transacción, una vivienda, un estudiante en una semana concreta— y **en qué momento** se hará la predicción. Esta decisión parece menor, pero condiciona qué datos se pueden usar y si la predicción llegará a tiempo para actuar.

c) Objetivo del proyecto frente a salida del modelo

El **objetivo** es de negocio o institucional (reducir costos, priorizar recursos, intervenir a tiempo). La **salida** del modelo es algo más modesto: una clase, un número, una probabilidad o una alerta. El modelo produce una *señal*; el impacto depende de la acción que alguien tome con ella. Si no sabemos qué haremos con la predicción, el problema todavía no está bien definido.

d) Criterio de éxito y costo de los errores

Conviene decidir desde el inicio **cómo sabremos si el modelo ayuda**, y reconocer que no todos los errores cuestan lo mismo: no detectar un caso grave suele ser peor que emitir una falsa alarma; sobreestimar la demanda genera exceso de stock y subestimarla, faltantes. Estos costos orientan después la elección de la métrica.

EJEMPLO • Deserción estudiantil en la carrera

Necesidad: «queremos reducir el abandono en primer año».

Tarea: clasificación binaria — estimar la probabilidad de que un estudiante abandone antes de fin de semestre.

Unidad de análisis: estudiante por semana (no estudiante por semestre), para poder alertar a tiempo.

Momento de la predicción: semana 6; solo pueden usarse datos disponibles hasta esa semana (asistencia, primeros parciales, entregas).

Acción: tutoría personalizada para quienes superen cierto umbral de riesgo.

Costo de errores: no detectar a un estudiante en riesgo es más grave que ofrecer una tutoría innecesaria.

DEFINICIÓN • Problema bien definido

Una necesidad traducida a una tarea que puede resolverse con datos, que especifica **qué** se predice, **sobre qué unidad, en qué momento, cómo se evaluará y qué acción** se tomará con el resultado. Incluye también lo que el modelo *no* puede prometer.

3.3.3 Conseguir y explorar los datos

El machine learning no trabaja con intenciones sino con datos concretos. Conseguirlos no es solo descargar un CSV: es reunir información **relacionada con la tarea**, que represente los casos relevantes y que esté **disponible cuando se necesite**. En proyectos reales los datos suelen estar dispersos entre bases de datos, planillas y registros manuales, y reunirlos es una tarea considerable.

Una vez reunidos, empieza la **exploración** —las técnicas de EDA de la Unidad 2—. Un diagnóstico inicial responde, como mínimo, estas preguntas:

Aspecto	Preguntas de diagnóstico
Estructura	¿Cuántas filas y columnas? ¿Qué representa cada fila? ¿Qué significa cada columna?
Tipos	¿Numéricas, categóricas, fechas, texto? ¿Alguna columna numérica se leyó como texto?
Calidad	¿Hay faltantes, duplicados, valores imposibles o categorías escritas de varias formas?
Distribuciones	¿Hay valores extremos, ceros inesperados, categorías con muy pocos casos?
Variable objetivo	¿Está bien definida? ¿Está desbalanceada? ¿Se conoce <i>antes</i> o <i>después</i> del momento de predecir?
Representatividad	¿Qué región, período o población cubren los datos? ¿Qué parte de la realidad queda fuera?

Esta etapa es un **diálogo entre la pregunta y los datos**: puede revelar que el problema es inviable tal como se planteó, o abrir oportunidades no previstas. Detectar una limitación temprano es mucho mejor que construir un modelo sobre una base equivocada.

A lo largo del tema usaremos el dataset **Diabetes** de scikit-learn como ejemplo de regresión: 442 pacientes, 10 variables clínicas (ya estandarizadas) y como objetivo una medida de la progresión de la enfermedad un año después.

In [1]:

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_diabetes

df = load_diabetes(as_frame=True).frame
print(df.shape)
print(df.isna().sum().sum(), 'valores faltantes')
print(df['target'].describe().round(1))
```

Out [1]:

```
(442, 11)
0 valores faltantes
count    442.0
mean     152.1
std       77.1
min       25.0
25%       87.0
50%      140.5
75%      211.5
max       346.0
```

El diagnóstico es favorable: no hay faltantes y el objetivo varía entre 25 y 346 con media 152. Retén la **desviación estándar (≈ 77)**: será nuestra vara para juzgar si los errores del modelo son grandes o pequeños.

3.3.4 Elegir variables

Elegir variables es decidir **qué información recibirá el modelo como entrada (X)** y distinguirla de lo que queremos predecir (y). No todas las columnas sirven: algunas son redundantes, otras están demasiado incompletas y otras, peligrosamente, parecen muy predictivas porque contienen información del futuro.

DEFINICIÓN · Fuga de datos (data leakage)

Ocurre cuando el modelo recibe durante el entrenamiento información que **no estaría disponible en el momento real de predecir**. Produce métricas artificialmente altas y un modelo inútil en producción. La estudiaremos a fondo más adelante; por ahora basta con hacerse siempre una pregunta: *¿conoceré este dato en el instante en que el modelo deba decidir?*

Además de la fuga, al revisar cada candidata conviene considerar:

Identificadores. Un código de estudiante o un número de cliente no describe ninguna característica real del caso; usarlo invita al modelo a memorizar filas en lugar de aprender patrones.

Ingeniería de características. A veces la variable útil no está en la tabla, sino que se construye: el día de la semana a partir de una fecha, la razón cuota/ingreso, los días desde la última actividad. Representar los datos de forma más cercana al problema suele ayudar más que cambiar de algoritmo.

Más no siempre es mejor. Variables ruidosas, redundantes o muy incompletas pueden confundir al modelo. Elegir también es **descartar**.

Sesgos y ética. Algunas variables reflejan desigualdades históricas o funcionan como aproximaciones de características sensibles. Que una variable mejore la métrica no significa que sea apropiado usarla.

ADVERTENCIA · Pistas del futuro

En el ejemplo de deserción sería un error incluir la **nota final**, el **estado administrativo al cierre** o el **total de faltas del semestre**. Solo se conocen después del abandono: el modelo obtendría un rendimiento espectacular en el laboratorio y fallaría por completo en la semana 6.

El conocimiento del dominio es clave en esta etapa: el docente, el médico o el analista del negocio sabe qué variables tienen sentido y cuáles estarán disponibles en la práctica. Por eso, en proyectos reales, elegir variables suele ser una tarea colaborativa.

3.3.5 Separar datos de entrenamiento y prueba

Si evaluamos el modelo con los mismos ejemplos con los que aprendió, obtenemos una impresión demasiado optimista: sería como tomar un examen con las mismas preguntas de la práctica. Lo que nos interesa es la **generalización**, es decir, el desempeño frente a casos nuevos. Por eso dividimos el dataset en dos partes disjuntas:

$$D = D_{\text{train}} \cup D_{\text{test}} \quad D_{\text{train}} \cap D_{\text{test}} = \emptyset$$

El conjunto de **entrenamiento** se usa para ajustar los parámetros; el de **prueba** se guarda aparte y solo se usa al final, como si fueran datos del futuro. Proporciones habituales son **80/20** o **70/30**, pero no hay una regla fija: con pocos datos no conviene sacrificar demasiados para la prueba; con muchos, basta una fracción pequeña.

La forma de dividir debe imitar el uso real del modelo:

Datos temporales. Si el modelo predecirá el futuro, se entrena con lo antiguo y se prueba con lo reciente; mezclar al azar filtraría información del futuro.

Clases desbalanceadas. En clasificación conviene **estratificar** para que ambas partes conserven la proporción de cada clase (parámetro `stratify` de scikit-learn).

Dividir antes de preparar. Imputaciones, escalados o selección de variables deben calcularse solo con el conjunto de entrenamiento; de lo contrario la prueba deja de ser independiente. Los `Pipeline` de scikit-learn resuelven esto de forma sistemática.

In [2]:

```
from sklearn.model_selection import train_test_split

X = df.drop(columns='target')
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
print('Entrenamiento:', X_train.shape, ' Prueba:', X_test.shape)
```

Out [2]:

```
Entrenamiento: (353, 10) Prueba: (89, 10)
```

El parámetro `random_state` fija la semilla del reparto aleatorio para que el experimento sea **reproducible**: cualquiera que ejecute el notebook obtendrá la misma división.

3.3.6 Entrenar el modelo

Entrenar es permitir que un algoritmo observe los ejemplos de entrenamiento y **ajuste sus parámetros** para que sus predicciones se acerquen a los valores reales. Ya lo vimos en detalle en el tema 3.1: en regresión lineal, entrenar es encontrar `w` y `b` que minimizan el MSE, por ecuación normal o por descenso de gradiente.

La palabra «entrenar» puede engañar: el modelo no *entiende* la diabetes ni la deserción; ajusta una estructura matemática a los datos. Y aunque suele verse como el centro del proyecto, depende por completo de las etapas anteriores: ningún algoritmo corrige un problema mal definido, datos poco representativos o variables con fuga.

In [3]:

```
from sklearn.linear_model import LinearRegression

modelo = LinearRegression()
modelo.fit(X_train, y_train)      # solo datos de entrenamiento
```

NOTA • Un modelo entrenado es una hipótesis

Que `fit()` termine sin errores solo significa que el código funciona, no que el modelo sirva. Lo que obtenemos es una **hipótesis aprendida de los datos**, que todavía debe ser puesta a prueba. Tampoco es buena práctica probar algoritmos al azar hasta que uno dé un número alto: la elección debe guiarse por el tipo de problema, los datos, la necesidad de interpretar y el costo de los errores.

3.3.7 Evaluar resultados

Evaluar es comparar el comportamiento del modelo con un criterio de calidad **sobre el conjunto de prueba**. En regresión no preguntamos si acertó exactamente, sino qué tan lejos quedaron sus predicciones; en clasificación importan no solo los aciertos sino **qué tipos de error** comete. Las métricas se estudiarán en detalle en unidades posteriores; aquí usamos tres de regresión:

$$\begin{aligned} \text{MAE} &= (1/m) \cdot \sum_i |\hat{y}^{(i)} - y^{(i)}| \\ \text{RMSE} &= \sqrt{(1/m) \cdot \sum_i (\hat{y}^{(i)} - y^{(i)})^2} \\ R^2 &= 1 - \sum_i (y^{(i)} - \hat{y}^{(i)})^2 / \sum_i (y^{(i)} - \bar{y})^2 \end{aligned}$$

MAE y **RMSE** están en las mismas unidades que `y`; **RMSE** castiga más los errores grandes. **R²** indica qué fracción de la variabilidad explica el modelo: 1 es perfecto y 0 equivale a predecir siempre la media.

a) Siempre comparar con una referencia

Antes de celebrar un resultado hay que preguntarse: ¿*mejora algo simple*? Una **línea base** (baseline) razonable en regresión es predecir siempre la media del entrenamiento; en clasificación, predecir siempre la clase más frecuente. Si el modelo no supera claramente a la línea base, no aporta valor.

In [4]:

```
from sklearn.dummy import DummyRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

base = DummyRegressor(strategy='mean').fit(X_train, y_train)

for nombre, m in [('Referencia (media)', base), ('Regresión lineal', modelo)]:
    p = m.predict(X_test)
    mae = mean_absolute_error(y_test, p)
    rmse = np.sqrt(mean_squared_error(y_test, p))
    r2 = r2_score(y_test, p)
    print(f'{nombre:<20} MAE={mae:6.1f} RMSE={rmse:6.1f} R²={r2:5.2f}')
```

Out [4]:

```
Referencia (media)  MAE=  64.0  RMSE=  73.2  R²=-0.01
Regresión lineal    MAE=  42.8  RMSE=  53.9  R²= 0.45
```

Lectura de los resultados

Modelo	MAE	RMSE	R²	Interpretación
Referencia (media)	64.0	73.2	-0.01	Ignora a cada paciente: no explica nada
Regresión lineal	42.8	53.9	0.45	Reduce el error en un tercio y explica ≈ 45 % de la variabilidad

El modelo **sí aporta** frente a la línea base. Pero un error típico de ~43 unidades, con una desviación estándar de 77, sigue siendo grande: sirve para señalar tendencias, no para decisiones individuales precisas. *Un resultado no es bueno o malo en abstracto; se juzga dentro del problema.*

ADVERTENCIA · La trampa de la métrica única

Con clases desbalanceadas, un clasificador que predice siempre «no abandona» puede acertar el 95 % de las veces y no detectar **ni un solo** caso de abandono. Por eso, además de calcular métricas, hay que revisar los errores concretos y comparar con una referencia. Y recordar que la evaluación inicial no es permanente: si cambian los datos o el contexto, el desempeño se degrada y debe **monitorearse**.

3.3.8 Mejorar el modelo

El primer modelo rara vez es el mejor; su papel es establecer una base de comparación. Mejorar **no** significa perseguir a ciegas un número más alto ni saltar a algoritmos más complejos: significa entender dónde falla el sistema y hacer cambios fundamentados.

Mirar los errores. ¿El modelo falla de manera uniforme o se concentra en ciertos grupos, rangos o períodos? Un modelo de tiempos de entrega que falla sobre todo en días de lluvia está diciendo qué información le falta.

Mejorar los datos. Etiquetas incorrectas, duplicados o columnas mal interpretadas se corrigen en los datos, no en el algoritmo.

Mejorar las variables. Nuevas características pueden aportar la señal que falta, siempre verificando que no introduzcan fuga.

Ajustar o cambiar el modelo. Los hiperparámetros y otros algoritmos se verán más adelante; al comparar, pesan también la interpretabilidad, la estabilidad y el costo de mantenimiento.

ADVERTENCIA · No ajustar mirando la prueba

Si repetimos «cambiar algo → medir en test» muchas veces, terminamos adaptando nuestras decisiones al conjunto de prueba y este deja de ser una evaluación honesta. Los ajustes se validan con datos de entrenamiento (por ejemplo, con **validación cruzada**, tema posterior); el test se reserva para la evaluación final.

Cada iteración debería partir de una **hipótesis** («creo que faltan variables de contexto») y quedar **registrada**: qué cambió, qué resultado produjo. Sin ese registro es fácil perderse entre versiones. Y a veces la mejora más honesta es reconocer que el modelo no está listo, o que machine learning no es el enfoque adecuado.

3.3.9 Comunicar conclusiones

Un proyecto no termina cuando la métrica es aceptable, sino cuando podemos explicar **qué se hizo, qué se encontró, qué tan confiable es y cómo debe usarse**. La comunicación debe volver siempre a la pregunta inicial. Un informe de proyecto de ML suele incluir:

Componente	Qué debe contener
Problema	La necesidad original y cómo se tradujo a una tarea de ML
Datos	Fuente, qué representa cada fila, período cubierto, variables principales y lo que faltó
Enfoque	Variables usadas, forma de separar los datos, modelos comparados y línea base
Resultados	Métricas interpretadas en unidades del problema y dónde se equivoca más el modelo
Limitaciones	Contextos donde no debe usarse, incertidumbre, dependencia de datos futuros
Recomendaciones	Cómo usarlo (p. ej., priorizar revisión humana) y qué mejorar antes de desplegarlo

El lenguaje debe adaptarse a la audiencia —no es lo mismo un equipo técnico que una dirección de carrera— sin perder rigor. Y debe ser **prudente**: un modelo aprende asociaciones en los datos disponibles; no «descubre la verdad» ni demuestra causalidad.

Evitar	Preferir
«El modelo demuestra que...»	«Los datos muestran una asociación entre...»
«Predice con certeza quién abandonará»	«Estima un mayor riesgo según las variables disponibles»
«El modelo tiene buen desempeño»	«Supera a la línea base y reduce el error medio de 64 a 43 unidades»

La comunicación cumple además una función de **documentación**: permite que otra persona reproduzca, audite o mejore el proyecto meses después.

3.3.10 Errores comunes al comenzar

Casi todos los errores de principiante tienen la misma raíz: avanzar demasiado rápido hacia el modelo y demasiado poco hacia la comprensión del problema.

Error	Consecuencia	Cómo evitarlo
Empezar por el algoritmo	Solución técnica sin pregunta clara	Formular la tarea, la unidad y la acción primero
Creer que tener datos basta	Modelo sin señal útil o sesgado	Diagnosticar calidad y representatividad
Usar información del futuro	Métricas infladas, fracaso en producción	Revisar cuándo se conoce cada variable
Evaluar con datos de entrenamiento	Optimismo engañoso	Reservar un conjunto de prueba
Confiar en una sola métrica	Errores críticos ocultos	Analizar errores y comparar con línea base
Suponer que más complejo es mejor	Sobreajuste y pérdida de interpretabilidad	Empezar simple y justificar cada salto
Ignorar el contexto de uso	Alertas que nadie revisa o decisiones injustas	Definir quién usa el modelo y cómo
No documentar	Resultados irreproducibles	Registrar decisiones y versiones
Comunicar con exceso de seguridad	Confianza indebida	Explicitar límites e incertidumbre
Usar ML cuando no hace falta	Complejidad sin beneficio	Considerar reglas simples o análisis descriptivo

NOTA • Síntesis del tema

Un proyecto de machine learning recorre ocho etapas: **definir** el problema, **conseguir y explorar** los datos, **elegir** variables, **separar** entrenamiento y prueba, **entrenar**, **evaluar**, **mejorar** y **comunicar**. El ciclo es iterativo y cada etapa protege contra un error distinto. Respetarlo no garantiza el éxito, pero sí aumenta mucho la probabilidad de obtener modelos útiles, evaluables y responsables.

Referencia: Palazzesi, A. (2026). *Ciencia de Datos con Python — Tomo 3: Machine Learning*, cap. 3 «El flujo general de un proyecto de Machine Learning». vintabytes.github.io